

Program 1:

```
crazyguy64@CrazyGuy64: ~/skill2
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main(){
printf("My process ID (PID) = %d\n", getpid());

return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill2$ nano program1.c
crazyguy64@CrazyGuy64:~/skill2$ gcc program1.c -o program1
crazyguy64@CrazyGuy64:~/skill2$ ./program1
My process ID (PID) = 717
```

Program2:

```
crazyguy64@CrazyGuy64: ~/skill2
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main(){
pid_t pid = fork();

if(pid == 0){
printf("Child Process\n");
printf("PID = %d\n", getpid());
}else{
printf("Parent Process\n");
printf("PID = %d\n", getpid());
}

return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill2$ nano program1.c
crazyguy64@CrazyGuy64:~/skill2$ nano program2.c
crazyguy64@CrazyGuy64:~/skill2$ gcc program2.c -o program2
crazyguy64@CrazyGuy64:~/skill2$ ./program1
My process ID (PID) = 793
crazyguy64@CrazyGuy64:~/skill2$ ./program2
Parent Process
PID = 803
Child Process
PID = 804
```

Program 3:

```
crazyguy64@CrazyGuy64: ~/skill2
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
    pid_t pid = fork();

    if(pid == 0){
        printf("Child Pprocess Executing...\n");
        sleep(3);
        printf("Child Finished\n");
    }else{
        wait(NULL);
        printf("Parent Process Continues\n");
    }
    return 0;
}
```

Output:

```

| print
crazyguy64@CrazyGuy64:~/skill2$ nano program3.c
crazyguy64@CrazyGuy64:~/skill2$ gcc program3.c -o program3
crazyguy64@CrazyGuy64:~/skill2$ ./program3
Child Pprocess Executing...
Child Finished
Parent Process Continues
crazyguy64@CrazyGuy64:~/skill2$
```

Program 4:

```

crazyguy64@CrazyGuy64: ~/skill2
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main(){
for(int i=1;i<=3;i++){
if(fork()==0){
printf("Child %d PID = %d\n", i, getpid());
return 0;
}
}
return 0;
}
```

Output:

```
crazyguy64@crazyguy64:~/skill2$ nano program1.c
crazyguy64@CrazyGuy64:~/skill2$ gcc program4.c -o program4
crazyguy64@CrazyGuy64:~/skill2$ ./program4
Child 1 PID = 674
Child 2 PID = 675
Child 3 PID = 676
```

Program 5:

```
crazyguy64@CrazyGuy64: ~/skill2
GNU nano 8.7.1
#include<stdio.h>
#include<unistd.h>

int main(){
printf("Before exec()\n");

execlp("ls","ls","-l",NULL);

printf("This will not execute.\n");

return 0;
}
```

Output:

```

crazyguy64@CrazyGuy64:~/skill2$ nano program5.c
crazyguy64@CrazyGuy64:~/skill2$ gcc program5.c -o program5
crazyguy64@CrazyGuy64:~/skill2$ ./program5
Before exec()
total 100
-rwxr-xr-x 1 crazyguy64 crazyguy64 16000 Aug 21 05:17 program1
-rw-r--r-- 1 crazyguy64 crazyguy64 113 Aug 21 05:17 program1.c
-rwxr-xr-x 1 crazyguy64 crazyguy64 16080 Aug 21 05:23 program2
-rw-r--r-- 1 crazyguy64 crazyguy64 228 Aug 21 05:23 program2.c
-rwxr-xr-x 1 crazyguy64 crazyguy64 16080 Aug 21 05:51 program3
-rw-r--r-- 1 crazyguy64 crazyguy64 259 Aug 21 05:51 program3.c
-rwxr-xr-x 1 crazyguy64 crazyguy64 16040 Aug 21 05:50 program4
-rw-r--r-- 1 crazyguy64 crazyguy64 160 Aug 21 05:14 program4.c
-rwxr-xr-x 1 crazyguy64 crazyguy64 16000 Aug 21 05:54 program5
-rw-r--r-- 1 crazyguy64 crazyguy64 157 Aug 21 05:54 program5.c
crazyguy64@CrazyGuy64:~/skill2$

```

Program 6:

```

crazyguy64@CrazyGuy64: ~/skill2
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main(){
fork();

for(int i=1;i<=5;i++){
printf("PID: %d Count: %d\n", getpid(), i);
sleep(1);
}
return 0;
}

```

Output:

```
crazyguy64@CrazyGuy64:~/skill2$ nano program6.c
crazyguy64@CrazyGuy64:~/skill2$ gcc program6.c -o program6
crazyguy64@CrazyGuy64:~/skill2$ ./program6
PID: 1060 Count: 1
PID: 1061 Count: 1
PID: 1060 Count: 2
PID: 1061 Count: 2
PID: 1060 Count: 3
PID: 1061 Count: 3
PID: 1060 Count: 4
PID: 1061 Count: 4
PID: 1060 Count: 5
PID: 1061 Count: 5
```

Program 7:

```
crazyguy64@CrazyGuy64: ~/skill2
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main(){
pid_t pid = fork();

if (pid == 0){
printf("Child Exiting\n");
return 0;
}else{
sleep(15);
printf("Parent Finished\n");
}
return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill2$ nano program7.c
crazyguy64@CrazyGuy64:~/skill2$ gcc program7.c -o program7
crazyguy64@CrazyGuy64:~/skill2$ ./program7
Child Exiting
Parent Finished
crazyguy64@CrazyGuy64:~/skill2$
```

Program 8:

```
crazyguy64@CrazyGuy64: ~/skill2
GNU nano 8.7.1
#include<stdio.h>
#include <unistd.h>

int main(){
    pid_t pid = fork();
    if(pid > 0){
        printf("Parent Exiting\n");
        return 0;
    }else{
        sleep(5);
        printf("Child PID: %d\n", getpid());
        printf("New parenr PID: %d\n", getppid());
    }
    return 0;
}
```

Output:

```
crazyguy64@CrazyGuy64:~/skill2$ nano program8.c
crazyguy64@CrazyGuy64:~/skill2$ nano program8.c
crazyguy64@CrazyGuy64:~/skill2$ gcc program8.c -o program8
crazyguy64@CrazyGuy64:~/skill2$ ./program8
Parent Exiting
crazyguy64@CrazyGuy64:~/skill2$ Child PID: 1219
New parenr PID: 326
```

